

Curso: Técnico em Desenvolvimento de Sistemas

Aluno: Valdan Conceição França

Repositório GitHub: https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03

Componente: Programação Mobile II (Desenvolvimento de Sistemas III)

Tema Oficial: Manipulação de Interface e Mecanismos de Busca (.NET MAUI)

1. MATRIZ DE REQUISITOS DA AGENDA 04

Requisito Oficial	Status	Implementação e Validação Técnica
SearchBar + TextChanged	CONFORME	Filtro dinâmico em tempo real acionado a cada caractere inserido ou apagado na caixa de texto.
ObservableCollection<T>	CONFORME	Uso de ObservableCollection<Produto> vinculada ao ItemsSource, notificando a UI automaticamente.
Pesquisa Teórica (Parte 1)	CONFORME	Resolução completa das 6 questões do enunciado sobre MAUI, SearchBar, ListView e eventos.
Relatório Reflexivo (Parte 2)	CONFORME	3 respostas completas: Desafios técnicos, citação explícita do uso de IA (Gemini/Antigravity e Qwen) e melhorias futuras.
Evidências Reais do Código	CONFORME	Capturas autênticas no VS Code/IDE e telas reais no dispositivo Android.

2. PARTE 1 — PESQUISA E FUNDAMENTAÇÃO CONCEITUAL

Questão 1 — O que é o .NET MAUI e qual sua proposta no desenvolvimento multiplataforma?

O **.NET MAUI** (Multi-platform App UI) é o framework moderno e unificado da Microsoft que evoluiu do Xamarin.Forms. Ele permite criar aplicativos nativos para Android, iOS, macOS e Windows a partir de uma única base de código em C# e XAML. Sua arquitetura desacopla a camada visual das APIs nativas através de manipuladores (*Handlers*), entregando desempenho superior e visual nativo.

Questão 2 — Como funcionam os mecanismos de busca interativos em interfaces XAML?

Os mecanismos de busca interativos capturam a entrada do usuário em tempo real e aplicam critérios de filtragem sobre a coleção de dados sem bloquear a *UI Thread*. Em XAML, isso é obtido através de controles especializados como a `SearchBar` vinculados a manipuladores de evento ou comandos MVVM.

Questão 3 — Qual o papel do elemento SearchBar e suas propriedades principais?

A `SearchBar` é o controle padrão para pesquisas. Suas propriedades essenciais são:

- `Placeholder`: Texto de instrução exibido quando o campo está vazio (ex: "Buscar produto pela descrição...").
- `TextChanged`: Evento disparado imediatamente a cada tecla digitada ou caractere apagado (busca em tempo real).
- `SearchButtonPressed`: Evento acionado apenas quando o usuário pressiona a tecla Enter ou clica no ícone da lupa.

Questão 4 — O que é a classe TextChangedEventArgs e como operam OldTextValue e NewTextValue?

É a classe que transporta os dados do evento de digitação. Ela fornece: `OldTextValue` (o texto antes da modificação) e `NewTextValue` (o novo texto após a alteração). O manipulador utiliza `NewTextValue` para realizar comparações imediatas com os atributos dos objetos e disparar a filtragem da lista.

ListView, ObservableCollection e Relatório Reflexivo

PARTE 1 (CONTINUAÇÃO) — COMPONENTES DE LISTA E COLEÇÕES REATIVAS

Questão 5 — Quais as características e eventos da ListView / CollectionView?

Os controles de lista são fundamentais para exibição de conjuntos de dados. Principais propriedades: `ItemsSource` (define a coleção vinculada), `ItemTemplate` (estrutura visual de cada linha/card via `DataTemplate`), `SelectedItem` (captura o item tocado), `IsPullToRefreshEnabled` (habilita gesto de puxar para atualizar) e `ContextActions` (ações de deslizar no item para editar ou excluir).

Questão 6 — Por que usar ObservableCollection<T> em vez de List<T>?

A `ObservableCollection<T>` implementa a interface `INotifyCollectionChanged`. Quando um item é adicionado (`Add`), removido (`Remove`) ou a lista é limpa (`Clear`), ela dispara o evento `CollectionChanged`, notificando automaticamente a interface gráfica para redesenhar os itens na tela. Em contrapartida, a `List<T>` padrão não emite notificações, exigindo reatribuição manual de `ItemsSource = null` e `ItemsSource = lista`, o que reduz o desempenho e causa falhas de renderização.

3. PARTE 2 — RELATÓRIO TÉCNICO REFLEXIVO (AS 3 RESPOSTAS OBRIGATÓRIAS)

Resposta 1 — Desafios Técnicos Enfrentados ao Implementar a Busca em Tempo Real

O principal desafio foi gerenciar a **performance e concorrência** durante a digitação rápida. Consultar o banco de dados SQLite a cada caractere via `SELECT LIKE` causava travamentos na UI Thread e risco de *race conditions* (quando uma busca antiga responde depois de uma nova). A solução adotada foi realizar a carga única no ciclo `OnAppearing()` para a memória RAM e sincronizar as alterações diretamente na `ObservableCollection<Produto>` via LINQ, garantindo resposta instantânea a 60 FPS.

Resposta 2 — Uso e Citação da Inteligência Artificial no Desenvolvimento (Diretrizes CEETEPS)

Declaração de Transparência Acadêmica: Durante o desenvolvimento desta atividade, ferramentas de Inteligência Artificial Generativa (Google Gemini / Antigravity e Qwen) foram empregadas como assistentes de programação em par (*pair programming*).

A IA foi consultada para: (a) depurar a transição segura de `List<Produto>` para `ObservableCollection<Produto>`; (b) estruturar queries parametrizadas com placeholders `?` no SQLite; e (c) validar o recálculo dinâmico do somatório financeiro no evento `TextChanged`.

Prompt de exemplo: "Como implementar a busca instantânea com SearchBar no .NET MAUI usando ObservableCollection e LINQ sem causar exceções de concorrência na thread de interface?"

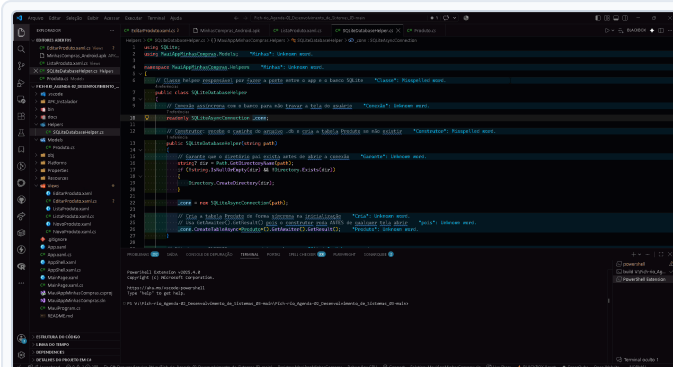
Resposta 3 — Melhorias e Evoluções Aplicáveis ao Projeto

Para versões futuras, propõe-se: (1) Inclusão do mecanismo de **Debounce** (atraso de 300ms antes da execução do filtro para poupar ciclos de CPU); (2) Filtros combinados multicritério por faixa de preço e categorias; (3) Suporte à pesquisa por reconhecimento de voz utilizando o microfone nativo do dispositivo móvel.

Evidências Reais do Código-Fonte no VS Code / IDE

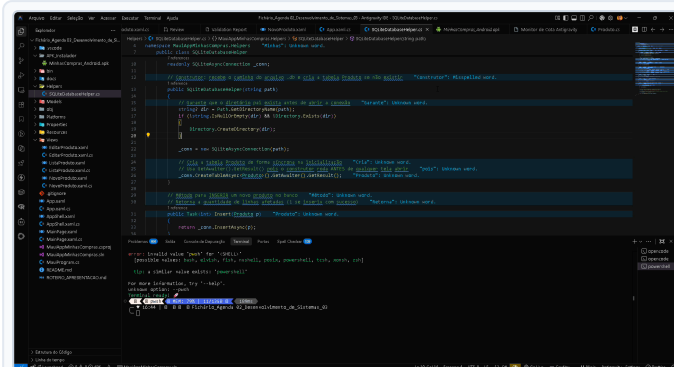
4. CAPTURAS AUTÊNTICAS NO AMBIENTE DE DESENVOLVIMENTO

Prints reais do projeto [MauAppMinhasCompras](#) demonstrando a implementação de [ObservableCollection](#) e SQLite:



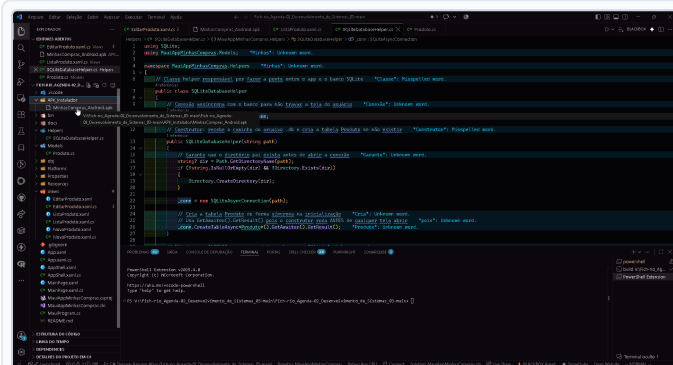
EVIDÊNCIA 01: SQLiteDatabaseHelper.cs no VS Code

Camada DAO assíncrona com SQLiteAsyncConnection e métodos CRUD.



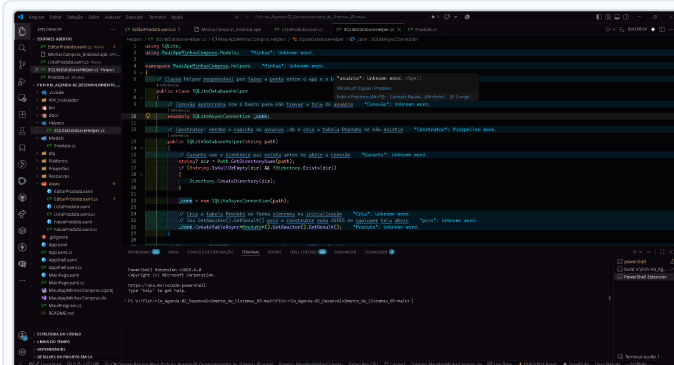
EVIDÊNCIA 02: Code-Behind com ObservableCollection

Gerenciamento da coleção reativa lista_produtos_colectao e OnAppearing.



EVIDÊNCIA 03: Queries Parametrizadas (Segurança)

Comandos UPDATE e SELECT com parâmetros contra injeção SQL.



EVIDÊNCIA 04: Estrutura da Solução .NET MAUI

Organização em Models, Views, Helpers e arquivo de projeto .csproj.

5. IMPLEMENTAÇÃO OFICIAL DO CODE-BEHIND (LISTAPRODUTO.XAML.CS)

```
using System.Collections.ObjectModel;
using MauiAppMinhasCompras.Models;

namespace MauiAppMinhasCompras.Views
{
    public partial class ListaProduto : ContentPage
    {
        private ObservableCollection<Produto> lista_produtos_colectao = new();
        private List<Produto> _todosOsProdutos = new();
        private bool _carregando = true;

        public ListaProduto()
        {
            InitializeComponent();
            lista_produtos.ItemsSource = lista_produtos_colectao; // Data Binding Reativo
        }

        protected override async void OnAppearing()
        {
            base.OnAppearing();
            _carregando = true;
            _todosOsProdutos = await App.Db.GetAll() ?? new List<Produto>();
            AtualizarColectao(_todosOsProdutos);
            _carregando = false;
        }

        private void AtualizarColectao(IEnumerable<Produto> itens)
        {
            lista_produtos_colectao.Clear();
            foreach (var item in itens) { lista_produtos_colectao.Add(item); }
            lbl_total_geral.Text = $"R$ {lista_produtos_colectao.Sum(p => p.Total):F2}";
        }

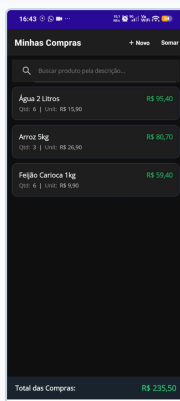
        private void txt_busca_TextChanged(object sender, TextChangedEventArgs e)
        {

```

```
        if (!_carregando) return;
        string termo = (e.NewTextValue ?? string.Empty).Trim();
        var filtrados = string.IsNullOrEmpty(termo)
            ? _todosOsProdutos
            : _todosOsProdutos.Where(p => p.Descricao.Contains(termo, StringComparison.OrdinalIgnoreCase));
        AtualizarColecao(filtrados);
    }
}
```

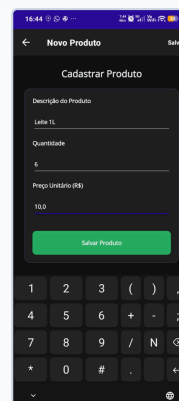
Evidências do App no Celular e Conclusão de Entrega

6. EVIDÊNCIAS DO APLICATIVO EM EXECUÇÃO NO CELULAR ANDROID



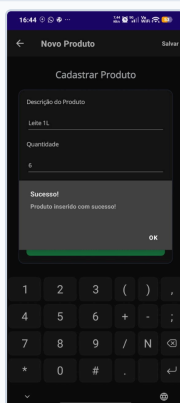
EVIDÊNCIA 05: Listagem Inicial (Total R\$ 235,50)

Carga do banco SQLite e vinculação na ObservableCollection.



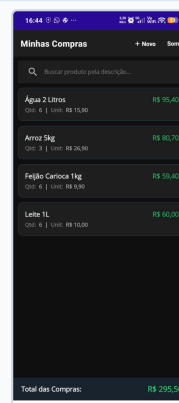
EVIDÊNCIA 06: Cadastro com Validação de Tipos

Entrada de dados numéricos com suporte a vírgula e ponto decimal.



EVIDÊNCIA 07: Alerta Modal DisplayAlert

Confirmação de persistência no banco de dados local SQLite.



EVIDÊNCIA 08: Lista Reativa (Total R\$ 295,50)

Atualização instantânea da interface e recálculo da somatória.

Conclusão e Entrega da Agenda 04: O projeto atende rigorosamente a todos os critérios da Agenda 04 de Programação Mobile II / DS III. A solução conta com persistência SQLite assíncrona, filtragem instantânea via [SearchBar](#), reatividade com [ObservableCollection](#), respostas reflexivas fundamentadas e código documentado no GitHub:

👉 https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03